

Experiment No. – 8

Aim –

(A.) A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

- a. The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.
- b. For every order, first calculate the **Gross Amount** using:
- c. **Gross Amount = Unit Price × Quantity**
- d. Then calculate the **Discount Amount** using:
- e. **Discount Amount = Gross Amount × Discount Percentage / 100**
- f. Finally, calculate the **Final Amount** using:
- g. **Final Amount = Gross Amount – Discount Amount**
- h. Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.
- i. The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

(B.) An e-commerce company wants to regularly analyze the performance of its **product categories**.

- a. The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple **JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements**.
- b. Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.
- c. Create a PostgreSQL **Materialized View** named: mv_category_sales

The report must contain:

- Category ID
- Category Name
- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

Tools Required

- Postgres
- PgAdmin4

Queries:

A.

```
CREATE TABLE customer_master(  
    customer_id VARCHAR(5) PRIMARY KEY,  
    full_name VARCHAR(50) NOT NULL,  
    city VARCHAR(30)  
);  
  
CREATE TABLE product_catalog(  
    product_id VARCHAR(5) PRIMARY KEY,  
    product_name VARCHAR(50) NOT NULL,  
    unit_price NUMERIC(10,2) NOT NULL  
);  
  
CREATE TABLE sales_orders(  
    order_id SERIAL PRIMARY KEY,  
    product_id VARCHAR(5)  
        REFERENCES product_catalog(product_id),  
  
    quantity INT NOT NULL,  
  
    customer_id VARCHAR(5)  
        REFERENCES customer_master(customer_id),  
  
    discount_percent NUMERIC(5,2),  
  
    order_date DATE NOT NULL  
);  
  
INSERT INTO customer_master  
(customer_id, full_name, city)
```

VALUES

```
('C1', 'Amit Sharma', 'Delhi'),  
( 'C2', 'Priya Verma', 'Mumbai'),  
( 'C3', 'Ravi Kumar', 'Bangalore'),  
( 'C4', 'Neha Singh', 'Kolkata'),  
( 'C5', 'Arjun Mehta', 'Hyderabad');
```

```
INSERT INTO product_catalog  
(product_id, product_name, unit_price)
```

VALUES

```
('P1', 'Smartphone X100', 25000),  
( 'P2', 'Laptop Pro 15', 65000),  
( 'P3', 'Wireless Earbuds', 5000),  
( 'P4', 'Smartwatch Fit', 30000),  
( 'P5', 'Gaming Console', 45000);
```

```
INSERT INTO sales_orders
```

```
(product_id, quantity, customer_id, discount_percent, order_date)
```

VALUES

```
('P1', 2, 'C1', 5, '2025-09-01'),  
( 'P2', 1, 'C2', 10, '2025-09-02'),  
( 'P3', 3, 'C3', 0, '2025-09-03'),  
( 'P4', 1, 'C4', 8, '2025-09-04'),  
( 'P5', 1, 'C5', 12, '2025-09-05'),  
( 'P1', 1, 'C2', 5, '2025-09-06'),  
( 'P3', 2, 'C1', 0, '2025-09-07');
```

```
CREATE OR REPLACE VIEW vW_PRODUCT_DISCOUNT_INFO
AS
SELECT
    O.order_id,
    O.order_date,
    C.full_name,
    P.product_name,

    (P.unit_price * O.quantity)
    -
    (
        (P.unit_price * O.quantity)
        * O.discount_percent / 100
    ) AS final_amount

FROM customer_master C

JOIN sales_orders O
    ON C.customer_id = O.customer_id

JOIN product_catalog P
    ON P.product_id = O.product_id;

EXPLAIN ANALYZE
SELECT * FROM vW_PRODUCT_DISCOUNT_INFO;

CREATE USER KRG_1A
PASSWORD '1234';

--GRANT AND REVOKE

GRANT SELECT ON vW_PRODUCT_DISCOUNT_INFO TO KRG_1A;
```

B.

```
CREATE OR REPLACE VIEW vW_PRODUCT_DISCOUNT_INFO
```

```
AS
```

```
SELECT
```

```
    O.order_id,
```

```
    O.order_date,
```

```
    C.full_name,
```

```
    P.product_name,
```

```
    (P.unit_price * O.quantity)
```

```
    -
```

```
    (
```

```
        (P.unit_price * O.quantity)
```

```
        * O.discount_percent / 100
```

```
    ) AS final_amount
```

```
FROM customer_master C
```

```
JOIN sales_orders O
```

```
    ON C.customer_id = O.customer_id
```

```
JOIN product_catalog P
```

```
    ON P.product_id = O.product_id;
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM vW_PRODUCT_DISCOUNT_INFO;
```

```
CREATE USER KRG_1A
```

```
PASSWORD '1234';
```

```
GRANT SELECT ON vW_PRODUCT_DISCOUNT_INFO TO KRG_1A;
```

Outputs:

Showing rows: 1 to 7						Page No: 1
	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	final_amount numeric	
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.000000000000000000	
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.000000000000000000	
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.000000000000000000	
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.000000000000000000	
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.000000000000000000	
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.000000000000000000	
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.000000000000000000	

Learning Outcome

After performing this SQL query, the following learning outcomes were achieved:

1. **Understand Complex Views – Learn how to create a view using multiple tables with JOIN operations.**
2. **Perform Calculations in Views – Learn to calculate derived values such as Gross Amount, Discount Amount, and Final Amount using SQL expressions.**
3. **Implement Data Security – Learn how views can restrict users from accessing sensitive or unnecessary columns from base tables.**
4. **Manage User Privileges – Learn to create PostgreSQL users and use GRANT and REVOKE to provide controlled access to database objects.**